

Introduction to Formal Methods

Christian Rinderknecht

Eszterházy Károly Katolikus Egyetem

September 2026

Introduction

Formal methods are tool-supported methods for software development that draw on concepts from theoretical computer science.

The reasons for using them are generally:

- reduced maintenance costs (particularly corrective maintenance),
- robustness (by explicitly delimiting the software's scope of validity),
- and reliability.

Their field of application is therefore, in particular, the dependability of critical or embedded systems, since the correctness and continuity of service of such systems may put at stake human lives or simply large sums of money.

Mastering complexity

The value of formal methods should not be underestimated. They have required decades of theoretical research as well as experimentation and are undergoing rapid expansion.

The increase in computing power and the drop in its price *de facto* contribute to the relevance of their application and sometimes even make their application simply possible in certain domains.

Some recent advances with AI agents (LeanZip) renew the hope of more affordable formal methods.

What is at stake here, indirectly, is the mastery of the size and logical complexity of computer systems, which is the central theme of software engineering.

The limits of computing

Inventors of perpetual motion machines make us smile because their inventions contradict our fundamental knowledge of physics: we know *a priori* that there is a flaw without needing to examine the details of the mechanism.

In computing there are achievements that ought to make us smile too. But which ones? Faced with a huge program that *almost* works, what should one do?

- Try to complete and correct it so that it works correctly and completely?
- Reconsider the design and start over with a different method for solving the problem?
- Conclude with amusement that the problem the program was supposed to solve is in fact not solvable by any program, this conclusion being drawn without a detailed examination of the program?

Limits and the engineer

Some limits on the use of formal methods are lifted by technical progress, but some limits are not due to electronics because they are intrinsic to mathematics, and hence to computation. It is thus possible to prove that **certain problems are not solvable and never will be**. Knowledge of these extrinsic and inherent limits is therefore part of formal methods.

Nowadays, the software industry strongly promotes pick-and-choose technologies and parades of buzzwords. This is why it is important for the computer engineer to understand the foundations of the discipline, which are independent of any fashionable technology: this way, he or she will be able to correctly **grasp tools in terms of the concepts** they involve.

Stakes

Public bodies have become aware of the progress made. In Europe (and in France since 1995), the ITSEC (*Information Technology Security Evaluation Criteria*) require the use of formal methods above a certain security level.

Along with a broadening of the fields of application (smart cards, highly secure information systems, robotics, electronic commerce, air traffic control, etc.), requirements have been rising.

In certain cases (transportation, control of nuclear power plants, medical computing) human lives are at stake.

Stakes

In telecommunications, the failure of an AT&T network in January 1990 caused considerable economic losses.

Currently, formal methods are applied to systems ranging from a sophisticated algorithm of a few pages to software of several tens of thousands of lines, *which is already quite useful*.

The explosion of flight 501 of the European Ariane rocket in 1995 and the failure of a Patriot surface-to-air missile (causing many deaths at a US military camp) are both due to software problems and showed the inadequacy of the methods then in use in the face of today's challenges.

Complex software is everywhere... and has bugs



Ariane 5. Credit: ignis, GFDL, CC BY-SA-2.5,2.0,1.0.

The case of Ariane 5

In June 1996, the maiden launch of the European rocket Ariane 5 ended in its self-destruction. The cause was in the software controlling the trajectory.

This piece of software was designed with the requirements of Ariane 4, **not** Ariane 5. Accordingly, some ranges of physical parameters were taken from Ariane 4, and wrongly assumed in the programming of Ariane 5.

Therefore, certain errors were deemed impossible for physical reasons and not handled in the code.

During the flight, a value overflowed, causing an exception to be raised, but not handled, which lead to the self-destruction of the rocket (the control system believed that it was off course).

The case of Ariane 5

The language used to program that piece of software in Ariane 5 was Ada.

Ada is a programming language originally designed in France in the early 80s, before it became the official language for most developments for the defence of the USA.

Ada is used a lot in critical embedded systems, deployed in transport, aerospace and defence.

The language was carefully designed so it lends itself well to all sorts of **static analyses**.

These analyses are static because they are performed by the Ada compiler itself, or by other tools that read the source code.

Their aim can vary, but perhaps the most important is **the detection of run-time errors** before the code is deployed.

The case of Ariane 5 / The after...math

The catastrophic failure of the first Ariane 5 incurred a very high cost, and the loss of ten years of research and development in satellite technology.

Some researchers at **Inria** (Gonthier & Deutsch) proposed a static analyser for Ada that could have caught the error that lead to the loss of Ariane 5, and discovered many other errors.

The mathematical theory they used to analyse the software is called **abstract interpretation**.

This powerful theory was invented in the 70s by the French researchers Patrick and Radhia Cousot.

PolySpace Technologies

In 1999, a start-up company called **PolySpace Technologies** was founded in France by Alain Deutsch (**Inria**) and Daniel Pilaud (co-inventor of Lustre, a declarative language for programming synchronous systems).

I joined that company in 2000 to work on the static analysis of JavaCard. The idea was to translate JavaCard and other languages (C and C++) to Ada, for which there was already a static analyser, following the explosion of Ariane 5. The translation did not need to preserve all behaviours, only those who could go wrong (run-time errors).

The company was a success and is now part of **Mathworks Inc.**, known for MATLAB and Simulink. PolySpace (the tool) is well known in the avionics industry.

Abstract interpretation

Abstract interpretation is a general framework to analyse programs and systems.

It makes an **over-approximation** of the behaviours of a program, then deduce properties of them, and finally translates back the abstract properties in terms of the original behaviours of the program.

Abstract interpretation is used to replace a system with a possibly infinite or very large number of states with a system that has a small number of states that can all be checked exhaustively.

Abstract interpretation can also be used to **prove the absence of errors**, or to find errors.

Abstract interpretation / An illustration

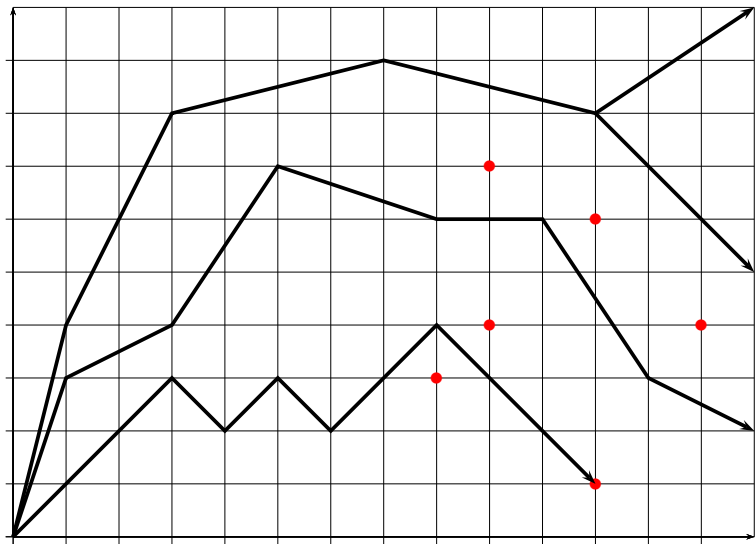
As an illustration, let us consider a system whose states are pairs of positive natural numbers, represented as coordinates on a quarter of plane.

A **state** is all the information characterising a system running at a given point in time: memory contents, program counter, files, network buffers, etc. (In the context of functional languages, the definition of a state is more restricted.)

Some states are coloured in **red** to denote that they are erroneous: this is the property 'To be erroneous'.

A behaviour of a system, or **trace**, can be thought as a line from one state to the next. We show four traces as black, thick lines. (One of the traces actually ends in an error.)

Abstract interpretation / An illustration



Abstract interpretation / An illustration

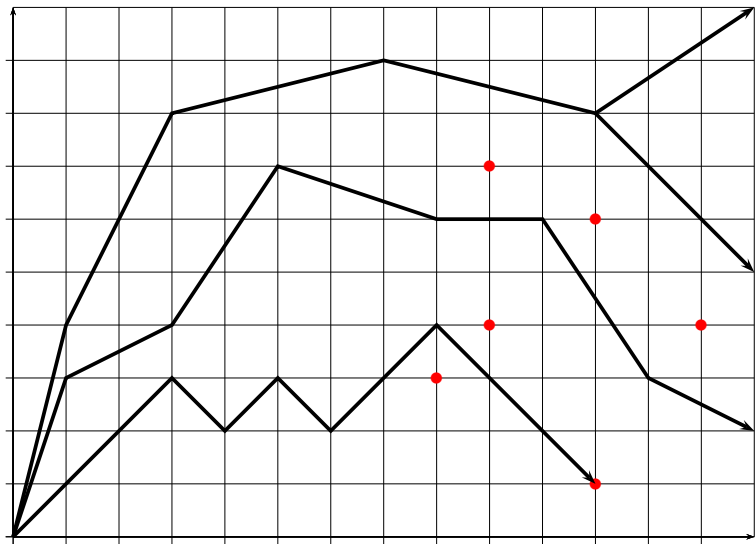
We need to characterise the erroneous states in a simple manner, even if we over-approximate them: this means to abstract the property 'To be erroneous.' so it becomes easy to check.

For example, we can abstract the error property by a red square containing the set of erroneous states, instead of being the exact, scattered set itself.

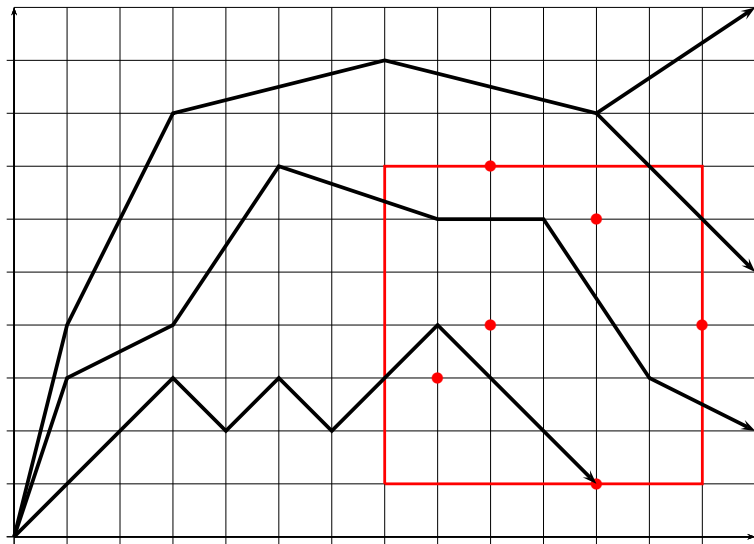
A square can be defined by a tiny amount of numbers, and it is easy to check whether a given state belongs to it: if so, we deem it an error.

In our example, this creates **false positives**, that is, states inside the square that are not actual errors.

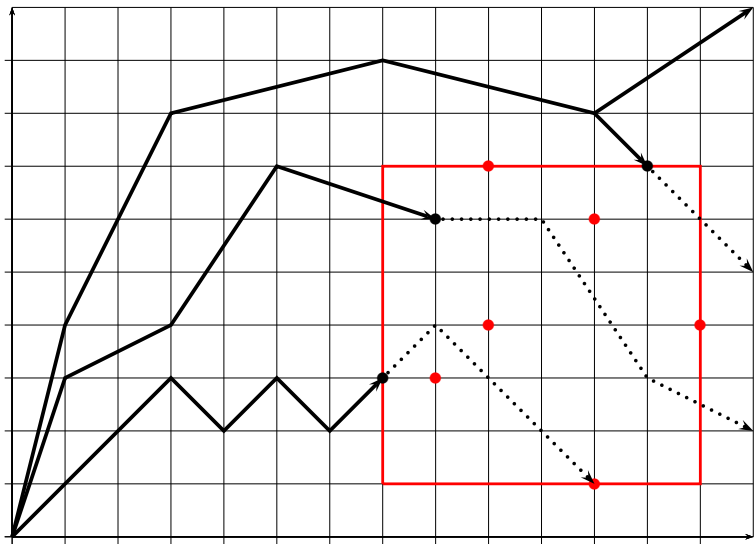
Abstract interpretation / An illustration



Abstract interpretation / An illustration



Abstract interpretation / An illustration



Abstract interpretation / An illustration

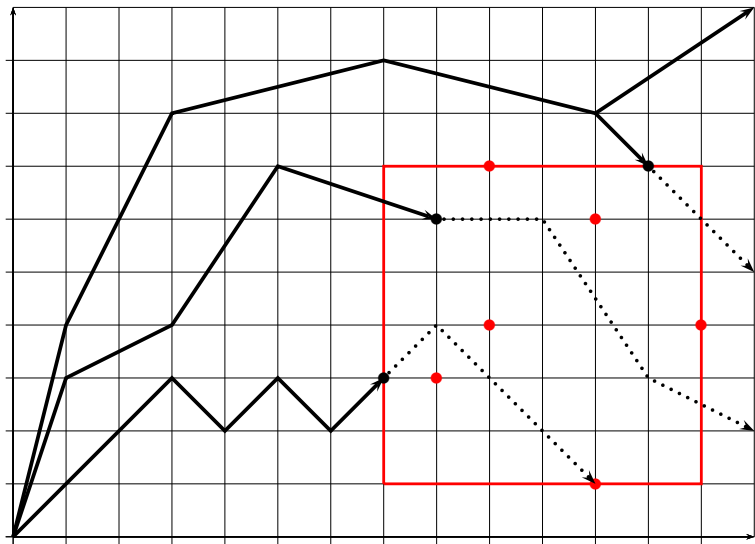
To reduce the number of false positives, we **refine the abstract property** so it fits better the erroneous states.

For instance, we could use another polygon to cover more tightly the erroneous states.

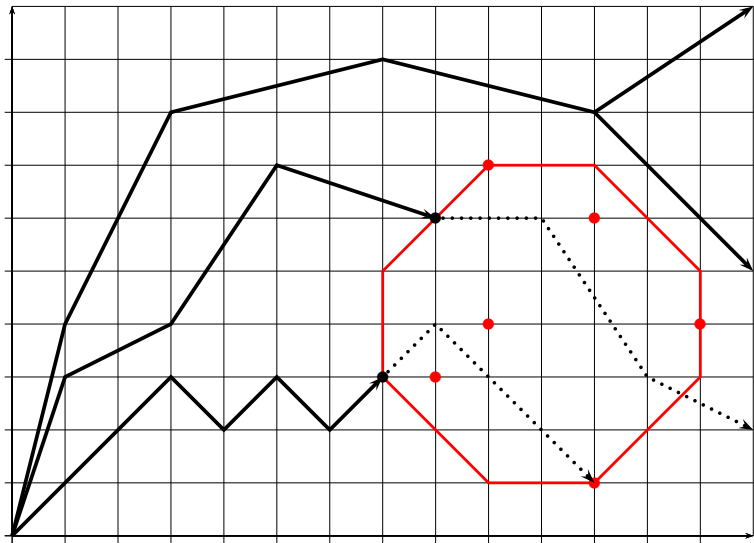
Let us try an hexagon: to define it, we need the same amount of information as for a square, but the inclusion property is slightly more complex.

This is better, but one trace remains a false positive. In general, false positives are inevitable if the property is complex enough, so trade-offs are necessary.

Abstract interpretation



Abstract interpretation





Airbus A340. Credit: Konstantin von Wedelstardt. GFDL 1.2

The static analyser Astrée

The usefulness of formal methods should not be underestimated. They have taken decades to mature from academic research to practical frameworks, and are becoming more and more widespread.

Astrée offers an abstract interpretation of C programs to detect run-time errors, in the same vein as PolySpace. It was designed at École Normale Supérieure (France) and is now commercialised by the German company AbsInt.

In 2003, Astrée proved automatically (on a standard PC) the absence of any run-time errors in the primary flight control software of the Airbus A340 fly-by-wire system: 132,000 lines of C. Astrée is written in OCaml and PolySpace in Standard ML, the American cousin of OCaml.

The static analyser Framac



Dassault Falcon 900. Credit: Aero Icarus, CC BY-SA-2.0

The static analyser Frama-C

Another well-known static analyser for C is **Frama-C**. It was designed by researchers at the CEA (France).

Frama-C is actually a software suite which includes a static analyser based on abstract interpretation, but also plug-ins to interactively prove that the source code satisfies some **functional specifications** provided by the user (like model checking).

Frama-C has been used by Dassault Aviation to detect common weaknesses in real-time software running on the Falcon jets.

The code base of Frama-C is almost entirely written in OCaml (perhaps the largest OCaml code base in the world).

Proof of critical railway software

To provide strong guarantees of software correctness, one must first properly specify the expected behaviour and delimit the scope of the proofs.

The use of the **B method** by GEC-Alsthom and, more recently, by Matra Transport International for the Calcutta metro (1992) and line 14 (1999) of the Paris metro illustrates this approach.

This method allows the **initial specification to be refined step by step** and provides a tool (based on set theory) for proving at each step the conformity (correctness) of the refinement with respect to the previous step. The final phase consists of producing several tens of thousands of lines of C code, for which one then has, by transitivity, a **proof of correctness with respect to the initial specification**.

A success story: the Parisian metro line 14



Métro line 14. Credit: Pline, CC BY-SA-3.0

The Parisian metro line 14

In October 1999, opened in Paris the first entirely automated metro line (number 14).

The trains move and stop based on decisions taken by themselves, with inputs from sensors and communication networks.

To ensure such a high-level of safety and continuity of service, the company Matra Transport used formal methods from the start of the project.

After the success of that new metro line, an existing metro line (number 1) has been automatised successfully.

Railways signalling systems are a major user of formal methods, due to the high safety requirements.

The Parisian metro line 14

The train system comprises four subsystems:

1. the automatic pilot and signalling system,
2. the operation control centre,
3. the platform doors,
4. the audio-video system.

Only the first subsystem includes software that is critical to the safety of the passengers.

It controls the movement of trains (speed, traction power, routes) and the doors (on-board and on the platforms).

It sends any alarm to the operation control centre.

The Parisian metro line 14

The system architecture is naturally distributed and is made of

1. the line equipment, in the operation control centre,
2. the wayside equipment along the tracks,
3. the on-board equipment.

They are interlinked via a high-speed communication network.

The Parisian metro line 14

In the operation control centre, operators monitor scheduled train programs for each day and supervise events received from the network.

Platform doors prevent people from falling on the tracks. The train doors open only when they are aligned with the platform doors.

If not properly aligned, there are emergency doors on the platform that can be manually opened.

Audio and video feeds allows operators to monitor the inside the carriages and to speak with passengers (a high-level of availability is required to ensure security).

The Parisian metro line 14

In order to bring strong correctness assurances in software, it is paramount to carefully specify the expected behaviours and delimit the scope of the mathematical proofs.

The use of the **B method** by Matra Transport International (now part of Siemens) for the Parisian metro line 14 is a case in point of this approach. (Another example is the Calcutta metro, in 1992.)

The B method was invented in the 80s by Jean-Raymond Abrial, a French researcher. It comprises

1. a formal notation to express specifications,
2. a design methodology,
3. and software tools.

The B method

The formal **B language** is based on **set theory**, which makes it very expressive. It enables the specification of a system and its expected properties into a model called an **abstract machine**.

It also enables the proof of such properties (like safety): some as **system invariants** (properties that must hold on all states), some as **postconditions** to mathematical functions.

A **proof assistant** then either automatically proves some of these theorems, or requests the engineer to prove them interactively.

The B method and its software tools allow the engineer to **refine** the abstract machine into a series of abstract machines until a **concrete machine** is obtained. A concrete machine is one that can be translated automatically into a programming language (C or Ada).

The B method

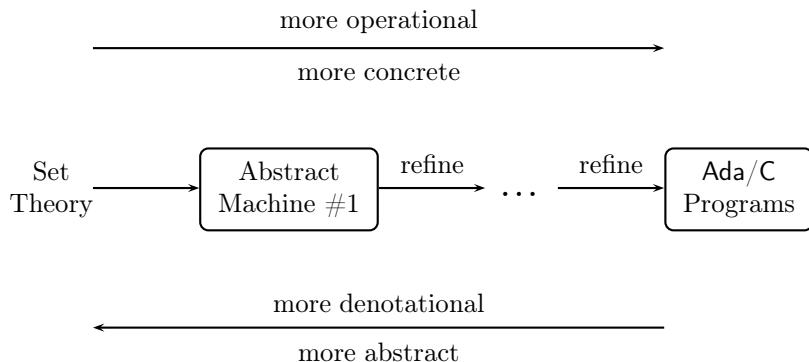
Each abstract machine is, in turn, less and less **denotational**, and more and more **operational**.

‘Operational’ means ‘what a system does’, whereas ‘denotational’ means ‘what properties the system satisfies’.

Step by step, the abstract machines become closer to **algorithms**, that is, functions operating over data structures, which is what a concrete machine is about.

The other aspect of a concrete machine is that it is **deterministic**: there are no unspecified choices left in the control flow (‘what to do next’ is always explicit).

The B method / Refinement



The B language / An abstract machine

An abstract machine in the B language consists of

- **states**, that is, variables satisfying some invariant properties, and
- **operations**, that is, functions that change the state while maintaining the invariants, and return values.

The variables making up the state of an abstract machine can denote mathematical sets, relations, functions, etc.

Operations are defined with

- **preconditions**: 'What is assumed about the state by the operation before starting', and
- **postconditions**: 'What can be assumed about the state and return value after the operation is finished'.

The B language / An abstract machine

Let us specify in the B language what is the Greatest Common Divisor (GCD) of two natural numbers.

In English: 'The GCD of x and y is d such that d divides both x and y , and any divisor of x and y is lower than or equal to d .'

In mathematical notation:

$$\text{gcd}(x, y) \mid x \wedge \text{gcd}(x, y) \mid y \wedge \forall z. (z \mid x \wedge z \mid y \Rightarrow z \leq \text{gcd}(x, y)).$$

The next slide shows the equivalent definition in the B language (note that variables must be at least two characters long).

Greatest Common Divisor (abstract)

MACHINE gcd /* Greatest Common Divisor */

OPERATIONS

rr <- gcd (xx,yy) =

BEGIN

PRE xx : INT & 1 <= xx & xx < MAXINT

yy : INT & 1 <= yy & yy < MAXINT

THEN

ANY dd **WHERE**

dd : INT

& (xx-(xx/dd)*dd = 0) & (yy-(yy/dd)*dd = 0)

& !zz.(zz : INT & zz < MAXINT

& (xx-(xx/zz)*zz = 0)

& (yy-(yy/zz)*zz = 0) => zz <= dd)

THEN rr := dd **END**

END

END

The B method / Refinement

The refinement from the initial abstract machine to the final concrete machine is stepwise and the engineer must provide more details about the low-level aspects of the system, whilst proving the new specification is **correct and complete** with respect to the previous one (all its behaviours are expected by the previous one, and vice-versa).

This transitively entails that the concrete machine is **equivalent** to the initial abstract machine.

The last step consists in the **generation of Ada programs** (or in C) which are thus, by construction, correct and complete with respect to the initial specification.

Greatest Common Divisor (concrete)

```
REFINEMENT gcd_R1 /* refinement of ... */
REFINES gcd /* ... the previous machine */
OPERATIONS
rr <- gcd (xx,yy) = /* No room to handle xx < yy */
BEGIN
  PRE xx : INT & 1 <= xx & xx < MAXINT
      yy : INT & 1 <= yy & yy < MAXINT
  VAR rem, old_xx, old_yy IN
  WHILE (yy != 0)
    old_xx := xold_yy := yyrem := xx - (xx/yy)*yy;
    xx := old_yyyy := rem
  INVARIANT gcd (old_xx, old_yy) = gcd (old_yy, rem)
  VARIANT yy < old_yy /* Termination */
  rr := xx
END
```

The B language / Ada

From the concrete machine, the following Ada code fragment would be produced, were the tooling told to inline the values of the variables `rr`, `old_xx` and `old_yy`, which are only needed for logical reasons (to state the invariant and variant, and the final result), not computational ones.

```
function GCD (Xx, Yy : Integer) return Integer is  
  Rem : Integer  
  while Yy /= 0 loop  
    Rem := Xx mod Yy  
    Xx := Yy  
    Yy := Rem  
  end loop  
  return Xx  
end GCD
```

Formal methods applied to compilers

Formal methods can also be applied to **compilers**.

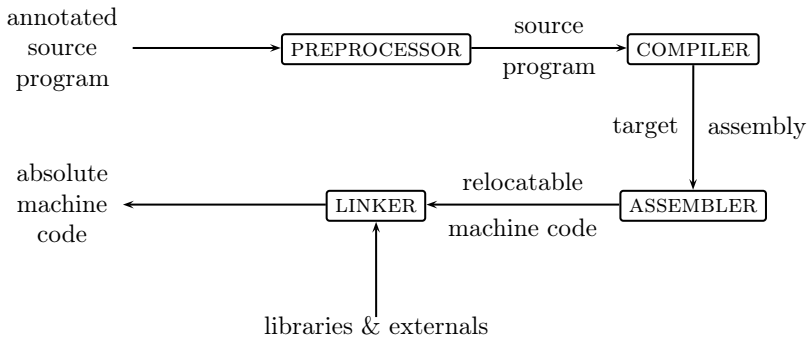
Compilers translate programs of high-level languages ('close to human thought'), like C, Ada or OCaml into programs of low-level languages ('close to machine thought'), like assembly or byte-code, and these two programs must be **operationally equivalent**.

This means that all behaviours of the input program must be preserved in the output program (**completeness**) and all behaviours of the output must map to behaviours of the input (**soundness**, also called *correctness*).

Since C is widespread in critical embedded software, it is desirable to consider C compilers.

General architecture of a C compiler

The compiler is one link in a **tool chain**:



General architecture of a C compiler

There are two phases to compilation: **analysis** and **synthesis**.

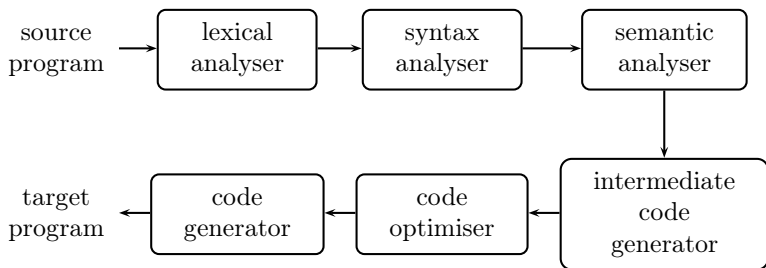
1. The analysis breaks up the source program into constituent pieces of an intermediary representation of the program.
2. The synthesis constructs the target program from this intermediary representation.

Actually, there can be many intermediary representations, each one getting closer to the target language.

Each phase is itself decomposable into other phases, depending on the source and target languages.

General architecture of a C compiler

The first row makes up the analysis and the second is the synthesis.



A real compiler is more complex than shown above, and errors are likely. Some errors can be silent: some erroneous code can be emitted only for certain classes of programs, which is a serious problem for embedded applications.

Xavier Leroy at **Inria** (France) lead the design and implementation of a **certified C compiler**, called **CompCert**, using Rocq.

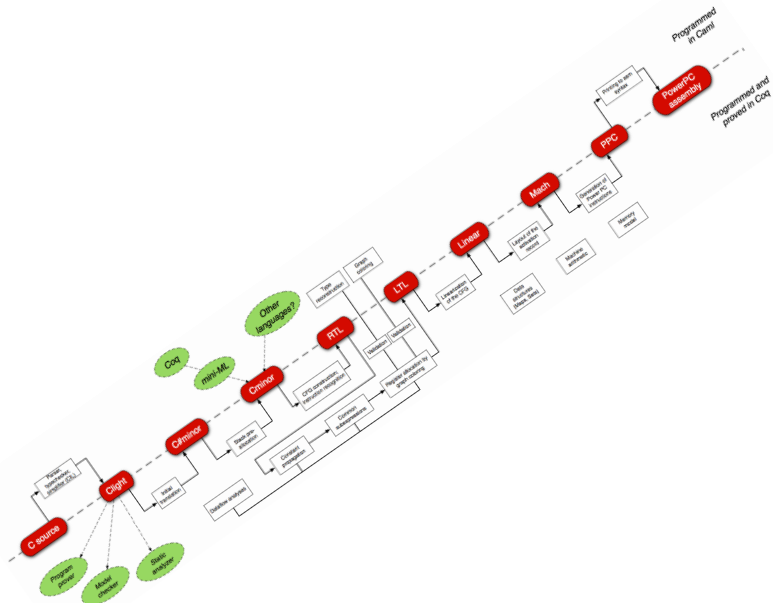
A formally verified C compiler: CompCert

CompCert is certified because its **OCaml code has been extracted from the specification** (of a large subset) of C and intermediary representations, as well as from **semantic-preserving transformations** between those representations, down to optimised assembly.

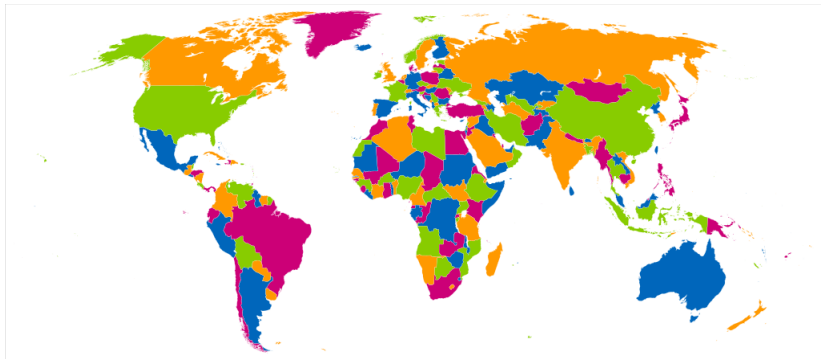
John Regher and his colleagues are experts in finding bugs in C compilers. They wrote, in 2011:

The striking thing about our CompCert results is that the middle-end bugs we found in all other compilers are absent. [...] This is not for lack of trying: we have devoted about six CPU-years to the task. The apparent unbreakability of CompCert supports a strong argument that developing compiler optimisations within a proof framework, where safety checks are explicit and machine-checked, has tangible benefits for compiler users.

A formally verified C compiler: CompCert



The proof assistant Rocq



Political map of the world. Credit: Fibonacci, CC BY-SA 3.0

The proof assistant Rocq

- Rocq is a **proof assistant**, that is, a piece of software in which to express logical formulas, prove them and have the proofs checked automatically. It is an interactive tool, but it automates the search of some proof patterns (libraries called **tactics**).
- Rocq comes with a programming language named **Gallina** which is close to OCaml, but **without side-effects** and adding **dependent types**. This type system is more general than the type systems found in usual languages like Java.
- Rocq features **code extraction**: it can generate an OCaml program from the proof of its properties.
- Code extraction relies upon the **Curry-Howard correspondence**, which states that a proof is analogous to a function, and the proved theorem is analogous to the type of that function.
- Rocq was created in 1989 by Huet and Coquand, at **Inria** (France), where it is developed today. It is written in OCaml.

Rocq: Other Applications

Here is a list of compilers certified by Rocq:

- CompCert, an optimising compiler for a subset of C by Xavier Leroy in 2012
- CertiCoq, a compiler by researchers at Princeton, Edinburgh, Inria and Cornell in 2017
- Q*cert, a query compiler by IBM Research in 2017
- Ergo, a compiler from Ergo (a language of legal smart contracts) to Michelson by Jérôme Siméon in 2018.

In graph theory, it was used to prove the **four colour theorem**, that states that countries in an arbitrary map can always be coloured with four colours (Gonthier, Microsoft Research, 2008).

Certified cryptographic primitives



Heartbleed bug logo. Credit: Codenomicon, CC0 1.0

Certified cryptographic primitives / Heartbleed

- In April 2014 was disclosed a bug in the OpenSSL cryptographic library introduced in 2012. The bug was named **Heartbleed** and given a logo to increase public awareness.
- The bug is due to improper input validation (missing bounds check) in the implementation of the TLS heartbeat extension and allows protected data to be read. The bug allows stealing private keys and users' session cookies and passwords
- The faulty code was written by a PhD student at Fachhochschule Münster, and the OpenSSL core developer that reviewed the code did not notice the problem.
- Around 500,000 SSL certificates were compromised by the bug and had to be reissued.
- Canada Revenue Agency was stolen 900 social insurance numbers in 2014 due to the bug. The records of 4.5 million patients were accessed after a breach into the system of Community Health, a private hospital in the USA.

Certified cryptographic primitives / HACL*

HACL*, is a formally verified cryptographic library, developed by the Prosecco team at **Inria** in collaboration with Microsoft Research.

HACL* is written in F*, an ML-style general-purpose functional programming language (with effects) aimed at program verification. F* is a cousin of OCaml and Rocq.

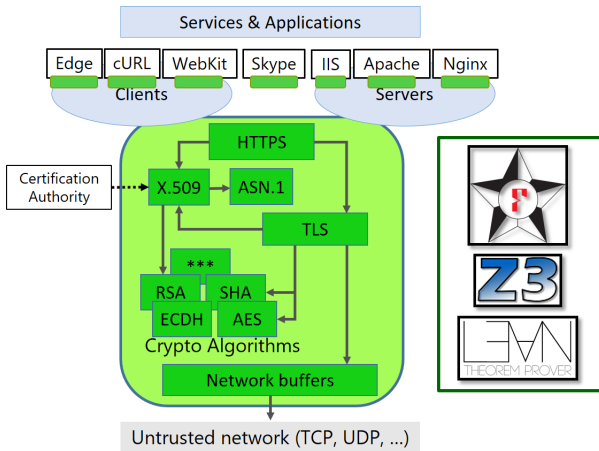
The F* code is then extracted into C with the KreMLin tool.

To generate C code that is proven is correct, the F* code is first transformed into Low*, a shallow embedding of a subset of C into F* with explicit memory management. The code is compiled from Low* to C and compiled with a C compiler (certified like CompCert, or not)

Certified cryptographic primitives / HACL*

According to the authors of HACL*,

when compiled with GCC on 64-bit platforms, our primitives are as fast as the fastest pure C implementations in OpenSSL and Libsodium, significantly faster than the reference C code in TweetNaCl, and between $1.1\times$ and $5.7\times$ slower than the fastest hand-optimised vectorised assembly code [...]



F* is an ML-style general-purpose functional programming language with effects aimed at program verification.

It was designed and developed at the joint Microsoft Research-Inria Centre.

F* programs can be extracted to efficient OCaml, F#, or C code. The F* type-checker aims at proving that programs meet their specifications using a combination of **SMT solving** and interactive proofs.

Like Rocq, F* can also be helped with intermediate lemmas and tactics when it cannot prove automatically the properties of the code.

There is an online tutorial
<https://www.fstar-lang.org/tutorial/>

Here is the factorial function in F*:

```
val fact: n:int{n ≥ 0} → Tot int
```

```
let rec fact n =  
  match n with  
    0 → 1  
  | _ → n * fact (n-1)
```

The annotation ' $n:\text{int}\{n \geq 0\} \rightarrow \text{Tot int}$ ' is requesting the F* compiler to prove that the factorial function terminates (Tot) and returns a positive integer ($\{n \geq 0\}$) when given as input a positive integer.

F* proves the function terminates using **well-founded recursion**.

If the positivity constraint in the input $n:\text{int}\{n \geq 0\}$ is removed, the type-checker of F* informs that it cannot prove the property.

Indeed, if $n < 0$, the function does not terminate and a stack overflow occurs.

The Intel Pentium bug



CPU Intel Pentium, 66 MHz. Credit: Konstantin Lanzet, CC BY-SA 3.0

The Intel Pentium bug

The floating-point unit (FPU) of some Pentium processors (Intel) contained an error when a certain kind of division was performed. The FPU is a numeric coprocessor, to which the main processor (CPU) delegates computations for a speed up (about $10\times$).

The error was discovered in 1994 by Thomas Nicely, a professor of mathematics at Lynchburg College (USA), who was performing number-theoretic calculations.

The division $1/824633702441.0$ is calculated incorrectly (all digits beyond the eighth significant digit are in error).'

— Thomas Nicely.

Intel had to recall all faulty units, incurring a large cost, but also a very bad press (CNN) in what was at the time a competitive market.

The Intel Pentium bug

That bug is known as the 'Intel Pentium FDIV bug', after the instruction for floating-point division.

Intel reacted by investing in formal methods for the design of its chips. For example, John Harrison (Intel) has formally verified the correctness (of rounding) of various floating-point algorithms using the **HOL Light prover** (a cousin of Rocq, also written in OCaml).

Also, various mathematical theorems in elementary number theory and real analysis were proved within the same framework.

Intel combined theorem proving with a more automated approach when the state space is finite and small enough: **model checking**.

Model checking

Model checking was invented in the 80s to check the properties of concurrent systems. (Clarke, Emerson, and Sifakis shared the 2007 Turing Award for their work.)

Model checking reasons on the **states of a program**.

It requires a way to define

- all possible states of a program, including those that should not be reached, and
- the execution of a program as the **transition** from one state to another (**traces**).

There is usually a very large, but **finite number of states**.

Model checking

Temporal logic is used to express properties of states

- **Safety**: a given ('bad') state can never be reached
- **Liveness**: a given ('good') state will eventually be reached
- **Reachability**: a given ('good') state can be reached.

States and sets of states are usually defined with **logical formulas**.

SMT solvers are often used to find a trace that leads to an undesirable state or prove the state is unreachable by exhausting all traces.

Temporal Logic

Reachability is a weak property, which is nonetheless useful to debug a model under construction, for example, by asserting that a process may access a given resource (a scenario can usually be automatically constructed).

Safety is a strong property reminiscent of a global invariant in non-temporal logics, and it can be utilised for example to state mutual exclusion, that is, that, at any time, there will be no more than one process accessing a common resource, or the absence of deadlocks, that is, that, at any time, there will be no processes waiting transitively on each other for the release of a lock on a shared resource.

Other interesting safety properties are the non-reception of unspecified messages and the absence of invalid end states.

Temporal Logic

In a temporal logic, doing nothing obviously realises safety, and this is where **liveness** comes into play, to require some progress. For example, liveness is useful for expressing the absence of starvation, in other words, the fact that a process will gain access to a resource.

In the theory of programming languages, **partial correctness**, by which, if a program terminates, then its output satisfies some condition, would be considered a safety property in temporal logics, whilst termination would be considered a liveness property.

A criterion to distinguish between safety and liveness is the following: safety can, in theory, be *disproved* in finite time, by simple observation of the states of the system execution, whereas liveness can be *proved* in finite time.

Temporal Logic

Let us note that temporal logics assume **infinite futures**, and since the system is finite, its behaviour must then be **cyclic**, as this happens to be the case of many communication systems implementing sessions or interactive loops, like servers and command shells.

This aspect is perhaps addressed more obviously by properties like **fairness**, which states that 'something good will happen an arbitrary number of times,' and can be conceived as a repeated liveness property. For example: 'If access to the critical section is infinitely often requested, then access will be granted infinitely often.' Fairness is most useful when dealing with the scheduling of processes.

The takeaway so far

- Bugs are everywhere and can be very costly.
- The **B method** helps manually narrowing the gap between specification and concrete implementation.
- **Abstract interpretation** runs an approximation of a program on an approximation of all possible inputs.
- Astrée, PolySpace and Frama-C are abstract interpreters written in OCaml, used in the aerospace industry.
- Rocq is a **proof assistant** used to check mathematical theorems.
- Rocq is also used to write **certified compilers**.

The takeaway so far

- F^* enables proving properties of functions using **type annotations**.
- F^* automatically proves the termination of recursive functions using **well-founded recursion**, but does not always succeed.
- F^* has been used to write efficient certified cryptographic libraries.
- **Model checking** is a technique to analyse all possible states of a program and prove that some (bad) states are unreachable.
- States and sets of states are defined with **logical formulas**.
- **SMT solvers** are used to find a trace reaching undesired states, or prove the absence of such a trace.

Communication and rigour

An often-overlooked benefit of rigorous or formal semantics is that it facilitates **communication** by providing an unambiguous framework for discussion and an impartial arbiter. It is also an excellent guide for building support tools (see compilation techniques). It is also sometimes possible to state the expected properties of a system and even to prove some of them.

Let us not forget that software maintenance consumes on average two thirds of the total cost of a project, and that fixing a specification error requires roughly twenty times more effort if it is only detected during operation.

Formal methods introduce additional **rigour** into software engineering in order to reduce these costs.

In 'formal method' there is 'method'

Software engineering shows that it is important to devote a great deal of effort to the early stages of the life cycle and, in particular, to establish reliable specifications, that is to say

- ones that correspond well to what is intuitively expected
- and that are consistent.

The first point rests on a good understanding of the client's needs, an understanding that the client cannot always articulate from the outset.

A **maieutic** process must therefore take place between the client and the designer in order to bring out the properties of the expected result. The designer builds a formal model step by step, which he or she does not necessarily share with the client but whose logical implications are discussed with the client in order to refine the properties.

In 'formal method' there is 'method'

The designer may also resort to **rapid prototyping**, which makes it possible to develop, in a short time, an easily modifiable version of the system that appears to be desired. The techniques used must then, above all, favour the responsiveness of the development process.

Considerations relating to the cleanliness or efficiency of the software can prove to be a handicap *at this stage*.

Once the prototyping stage is complete, priorities change and the goals of quality and rigour come back to the fore.

The use of functional languages, such as OCaml, simplifies the transition from prototype to product.

The formal language as a guiding thread

Formulating the problem to be solved in a formal or semi-formal language is the first step of the solution. Indeed, a formal language can be equipped with a sound semantics, especially if it is based on well-established mathematical theories.

It then becomes possible to prove properties of the system, on the one hand at the specification level and, on the other hand, at the code level, which can therefore be guaranteed to conform to the specification.

- **Correctness** means that all values returned by the program are indeed accounted for by the specification.
- **Completeness** means that the program does not omit any of them.

The formal language as a guiding thread

For reasons that touch on the foundations of mathematics, these properties, like others, may not be provable in general (this is called **undecidability**). This brings us back to the limits of computability.

There are several ways to address conformity (correctness): Hoare logic, enumeration of reachable states, refinement of specifications, rewriting, program calculation or extraction.

A formal language lends itself well to the development of tools that automatically assist with the various tasks.

Testing, maintenance, and sometimes coding effort are reduced, and control over the life cycle increases, as documentation becomes much more reliable.

Formal methods for the client

Formal methods are also relevant to organisations that outsource all or part of their software development to third parties. Indeed, the client organisation must ensure

1. that its specifications are correct,
2. that the delivered product matches the specifications.

For the second point, the client organisation must, at the very least, validate the product and, to do so, subjects it to a battery of tests.

The complexity of testing grows considerably with that of the product and its new versions. Also, automatic generation of test suites from formal specifications is achieving increasing success.

The limits of testing for the client

Testing, while always necessary, can prove insufficient, depending on the level of requirements, because it covers only a certain number of the system's behaviours. Structural coverage of all branches of the code does not account for all possible combinations of values (which may in theory be infinite).

Sometimes the client organisation supplies the developer with the test suite that the product must pass. This can prove problematic if the developer deliberately supplies a product that passes the tests (a finite number of cases) but is otherwise flawed.

If the product is built using formal methods, it can be delivered together with a proof of its conformity, which the client organisation can then verify. It is far easier to check a proof than to discover one.

A few caveats...

Formal methods are not a panacea: when facing complex problems there is no easy solution, only reasoned compromises.

One must keep in mind the gap that separates the formal specification from the reality it models. This gap can only be narrowed through a process of review, reformulation, confrontation with conjectures, and maieutic dialogue.

One obstacle to the use of formal methods is a lack of mathematical culture and familiarity with notation. In civil engineering or electronics, mathematics is part of the landscape.

... and some good news

Formal methods place heavy demands on the early phases of development (specification, design), at the risk of giving a negative impression of slowdown. But later phases (testing, integration) are shortened and better controlled.

Formalisation (using logic to model) reveals many tricky points earlier. These tricky points are often inherent to the problem to be solved, which may therefore be much more complex than it first appears.

The use of abstract notation is then in fact a plain necessity, not an added complication.

An overview of formal methods

Formal approaches present very varied aspects. Several families currently exist, most of them featuring

- a particularly prominent underlying theory (for example, transition systems, set theories, universal algebras, the λ -calculus),
- a favoured domain of application (for example, data processing, real time, protocols)
- and a community of researchers and users, sometimes split into several variants or schools.

Specialised or general approaches

Specifying a system covers several aspects, including architecture, observable behaviours, databases, algorithms, etc.

Some methods model systems from the perspective of data transformation, or flow, or state machines.

Information exchange can be modelled via shared data (centralised memory), synchronous communication (which assumes a single clock with no propagation delay) or asynchronous messaging, or function invocation (of the *Remote Procedure Call* kind in C or *Remote Method Invocation* in Java).

Specialised or general approaches

Other methods are less constraining and provide an excellent general interpretive framework.

Specialised approaches, by often favouring the operational aspect (tooling), can obscure the overall understanding and can struggle to keep up with changes to the object being studied.

General approaches, closer to mathematical logic, offer great freedom of expression but fall short on the methodological front.

Example: the role of states

A typical example of a difference in approach is the role given to the states of a system in its modelling.

This notion seems fundamental if one considers the execution support of software (computers or virtual machines), which presupposes a memory whose contents can change and, possibly, a clock.

On the other hand, this notion is not fundamental in mathematics, even though mathematics can speak of states. From a theoretical standpoint, it is trickier to describe the composition of instructions with effects on memory than that of purely mathematical functions.

Example: the role of states

This duality is handled differently depending on the formal technique.

For example, in the B method there is a notion of **simultaneous assignment**, whose benefit, by grouping a sequence of assignments that do not interfere with one another, is to reduce the number of states.

Moreover, **functional languages** offer a semantics with no notion of state, but also allow states to be used: it is left to the programmer to use states (referred to as **imperative features** in programming, such as loops and arrays) only in a peripheral way.

Favouring specification or verification

- A formal method is built from two main ingredients: a specification language and a verification system. These two elements are unevenly developed depending on the approach and the tools.
- Thus, the Boyer-Moore proof-assistance systems favour automation of proofs at the expense of ease of expression. In contrast, the design of Z aimed above all for expressive power, which led to a language that is difficult to tool.
- More recent approaches, such as HOL and Rocq, try to combine the best of both worlds: relying on powerful logics, they come with tools that, on the one hand, help construct proofs and, on the other hand, allow those proofs to be fully checked with a high degree of confidence.

Overview of logical tools

Mathematical logic has developed along several lines:

- model theory,
- proof theory,
- set theory,
- type theory,
- computability theory.

The importance of logic in the context of software engineering can be summarised in two points:

- it provides a framework for expressing many notions;
- it lends itself well to formalisation.

Logic as a framework

Regarding the first point:

- The data manipulated by programs can be described by combinations of elementary sets (integers, characters, etc.) such as Cartesian products (for struct in the C language) or unions (for union in C), for example.
- Computability theory teaches us of the existence of limits, that is, here, of unrealisable specifications.
- Type theory leads to compilers that guarantee the safety of programming (through static type checking and inference).

Logic for modelling

Regarding the second point, there are two benefits to formalising:

- the rigour of texts and reasoning is increased, since they can at least partly be reduced to interpretable manipulations of symbols;
- the resulting work can be assisted or, sometimes, automated — in all cases, tool-supported.

Model theory

There are essentially two complementary ways of specifying:

- representing a system by its properties,
- giving a constructive model of it.

This is sometimes referred to as **specification by properties or by models**. Properties are expressed by logical axioms and models by set-theoretic operations. The first case is called the syntactic aspect and the second the semantic aspect (since it describes the universe of discourse itself). For example, models satisfying the statement $\forall x. \exists y. (y R x)$ include $(\mathbb{N}, >)$ or $(\mathbb{R}, <)$ or $(\mathbb{N}, \text{'is a multiple of'})$, etc.

Semantic consequence is fundamental: a statement E is a semantic consequence of statements A, B, C , etc., if *every* model having properties A, B, C , etc., also has property E .

Proof theory

The semantic-consequence relation has the drawback of needing to be checked over all possible models, and there may in general be infinitely many of them.

One may then prefer a **provability relation**: a statement E is said to be provable (or demonstrable) from statements A, B, C , etc., if a formal proof of E can be constructed using only the hypotheses A, B, C , etc., together with the axioms and rules of logic. The statement E is said to be **refutable** if its negation is provable.

It is nevertheless necessary that formal manipulations preserve the semantics of statements: this is **soundness**. If every semantic consequence is provable, then the theory is said to be **complete**.

Proof theory

Independently of the links between the notions of semantic consequence and provability, there are questions intrinsic to the latter, such as: if E is provable, does there exist a proof of E using only subformulas of E ? If so, the search space is considerably reduced, which has an impact on proof assistance.

The study of axioms and rules as formal (that is, purely syntactic) calculi, together with their relationships to the notion of semantic consequence, constitutes **proof theory**.

Cantor's set theory and Russell's paradox

It is useful to form a set whose elements possess a given property — such a definition is said to be *by comprehension*. If this type of definition is left unconstrained, one ends up within the framework of the first, so-called naive, set theory, which turned out to be contradictory at the beginning of the 20th century.

A theory is contradictory if one can prove both a statement and its negation, and hence, in the end, anything at all.

Consider Russell's paradox. Let us form, by comprehension, the curious set $R = \{x \mid \neg(x \in x)\}$. If $R \in R$ then R must satisfy the property held by its elements, namely $\neg(R \in R)$. Conversely, if $\neg(R \in R)$ then R satisfies the defining property of the elements of R , that is, $R \in R$. Hence, setting the property $P = R \in R$, we have proven both P and $\neg P$, which is contradictory.

Zermelo-Fraenkel axiomatic set theory

One way to circumvent the pitfall of paradoxes is to reformulate set theory axiomatically (that is, with logical rules) by introducing a form of *stratification of constructions*. In this way, a property cannot be arbitrarily defined by ranging over an arbitrary property, as is the case in Russell's paradox.

This theory, proposed by Zermelo and Fraenkel, allowed for a new foundation of mathematics, and no one has reproduced a paradox within it to this day.

The specification languages Z and B rely on this theory, adding a touch of typing to bring it closer to the concerns of computer science.

Type theory

Another way to avoid paradoxes was proposed by Russell himself: type theory. This too involves stratifying sets (and properties): at level 0, individuals; at level 1, sets of individuals (and properties concerning individuals); at level 2, sets of sets of individuals, and so on. Types characterise these layers. They are associated with individuals and with basic properties and constrain the construction of the higher layers.

In computer science, this approach has proven very fruitful, as shown by many recent programming languages. To allow for recursion, a more permissive typing rule is needed. The unavoidable drawback is then having to allow programs that do not terminate.

Computability theory

Another fundamental aspect is the question of which mathematical functions are computable by machines: this is the subject of computability theory.

In the 1940s, several approaches were proposed:

- Turing machines,
- the λ -calculus (Church),
- partial recursive functions (Gödel, Herbrand).

Each provides a framework in which to express a notion of *effective procedure* (algorithm). The *Church thesis* is that these various formalisms are equivalent (they compute the same set of mathematical functions).

Countable sets

Are all functions computable? To answer this, a detour through the theory of formal languages proves useful.

A set is *countable* if it is finite or can be put in bijection with the set of natural numbers $\mathbb{N}$. The cardinality of a countably infinite set is denoted $\aleph_0$. For example

- The set of even numbers is countable (the trivial bijection is $2n \mapsto n$). In general, every infinite subset of the naturals is countable.
- The set of finite words over the alphabet $\{a, b\}$ is countable (the bijection is built by ordering the words lexicographically).
- The rational numbers are countable.
- Regular expressions are countable.

Cantor's diagonalisation

One may then ask whether there exist sets whose cardinality is greater than $\aleph_0$, that is uncountable sets. The answer is yes: **the set of subsets of a countable set is not countable.**

This is proven by a technique called **diagonalisation**. Let $A = \{a_0, a_1, \dots\}$ be a countable set and let S be the set of its subsets.

Let us argue by contradiction and assume that S is countable, and therefore that $S = \{s_0, s_1, \dots\}$.

Cantor's diagonalisation

Let us construct the following infinite table:

	a_0	a_1	a_2	a_3	a_4	$\dots$
s_0	×	×		×		
s_1	×	○		×		
s_2		×	×		×	
s_3	×		×	○		
s_4		×		×	○	
$\vdots$						$\ddots$

It indicates which elements of A belong to which elements of S : a cross at column i and row j indicates that $a_i \in s_j$.

Cantor's diagonalisation

Consider the set $D = \{a_i \mid \neg(a_i \in s_i)\}$. Its elements are marked with a circle in our table (on the diagonal). We have $D \subset A$, but is D actually listed among the s_j ?

Suppose there exists k such that $D = s_k$. But $a_k \in D \Leftrightarrow \neg(a_k \in s_k)$, which implies $D \neq s_k$, thus contradicting the assumption. Hence D is not listed among the subsets of A , even though it is a subset of A . This therefore contradicts the assumption that the set of subsets of a countable set is countable.

The cardinality of an uncountable set is denoted $2^{\aleph_0}$, by analogy with the fact that the cardinality of the set of subsets of a finite set of cardinality n is 2^n .

Undecidable problems

- The set of languages is the set of subsets of a countable set (the set of words), and therefore, by the previous theorem, it is not countable.
- The solution to a problem corresponds to a language, so the number of solutions (or problems) equals the number of languages, and is therefore not countable.
- An algorithm is a finite string, so the set of algorithms is countable.

There are therefore more solutions than computable solutions (solutions obtainable by algorithms). We speak of **decidable problems** for those whose solution is computable and of **undecidable problems** for those whose solution is not computable.